AWS STUDY NOTES

Amazon Route 53

Comprehensive Guide to DNS, Routing Policies, Health Checks & Security

Prepared: April 5, 2026 | Port 53: the DNS standard port | AWS Cloud Services

Table of Contents

1. Introduction
2. DNS Fundamentals
   - How DNS Works (Hierarchical System)
   - DNS Hierarchy
   - DNS Record Types
   - Time To Live (TTL)
   - DNS Caching Behavior
3. Route 53 Hosted Zones
   - Public vs. Private Hosted Zones
   - Hosted Zone Architecture
   - Split-View DNS
4. Routing Policies
   - Simple | Weighted | Latency-Based
   - Failover | Geolocation | Geoproximity
   - Multivalue Answer | Comparison Matrix
5. Health Checks
   - Endpoint | Calculated | CloudWatch Alarm
   - Health Check Locations & Best Practices
6. Alias Records vs. CNAME Records
7. Traffic Flow & Traffic Policies
8. DNSSEC (DNS Security Extensions)
9. Tips & Best Practices
10. Pitfalls & Remedies

1. Introduction

The **Domain Name System (DNS)** is the internet's phone book, translating human-readable domain names (e.g. `www.example.com`) into IP addresses (e.g. `192.0.2.1`) that computers use to communicate. Every internet interaction begins with DNS — when a user types a URL, their browser queries DNS servers to find the website's location.

Amazon Route 53 is AWS's highly available and scalable DNS service that goes far beyond basic name resolution. It offers intelligent routing policies, health checking, traffic management, and seamless integration with AWS services. The name "Route 53" references TCP/UDP **port 53**, the standard port for DNS.

Why Route 53 Matters

Traditional DNS provides simple domain-to-IP mapping; Route 53 transforms DNS into an intelligent traffic management platform:

- **Geolocation routing** — directs users to the nearest data center for lower latency.
- **Weighted routing** — enables gradual traffic migration between deployments.
- **Failover routing** — automatically switches to backup resources during outages.
- **Latency-based routing** — sends users to the fastest-responding region.

These capabilities make Route 53 essential for multi-region architectures, disaster recovery, blue-green deployments, and global application delivery.

> **COMMON MISTAKES TO AVOID**
>
> Misconfigured TTL values cause prolonged outages during failover. Improper health checks trigger false positives. Not using alias records for AWS resources incurs unnecessary costs. Missing DNSSEC leaves domains vulnerable to cache poisoning attacks.

2. DNS Fundamentals

2.1 How DNS Works (Hierarchical System)

DNS operates as a distributed hierarchical system. Below is the resolution flow from browser to authoritative server:

```
Step 1  User types www.example.com in browser
      |
Step 2  Browser checks local DNS cache
        If found: Use cached IP (fast path)
        If not:   Query DNS resolver
      |
Step 3  Recursive Resolver (ISP or 8.8.8.8)
        Checks its cache; if not cached, begins recursive query
      |
Step 4  Root Name Server (.)
        "I don't know www.example.com"
        "But ask .com TLD server at 192.5.6.30"
      |
Step 5  TLD Name Server (.com)
        "I don't know www.example.com"
        "But ask example.com authoritative server at 203.0.113.1"
      |
Step 6  Authoritative Name Server (Route 53)
        "www.example.com is at 198.51.100.1"
      |
Step 7  Response travels back through resolver to browser
      |
Step 8  Browser connects to 198.51.100.1
```

2.2 DNS Hierarchy

```
Root Zone (.)
├── Top-Level Domains (TLDs)
│   ├── Generic TLDs:      .com, .org, .net, .io
│   ├── Country Code TLDs: .uk, .de, .jp, .in
│   └── New TLDs:          .cloud, .tech, .app
└── Second-Level Domains
    ├── example.com
    │   ├── www.example.com   (subdomain)
    │   ├── api.example.com   (subdomain)
    │   └── mail.example.com  (subdomain)
    └── company.org
        ├── blog.company.org
        └── shop.company.org
```

2.3 DNS Record Types

A Record (Address Record)

- Maps domain to IPv4 address — e.g. `www.example.com → 192.0.2.1`
- Most common record type; required for website hosting.

AAAA Record (IPv6 Address)

- Maps domain to IPv6 address — e.g. `www.example.com → 2001:0db8::1`
- Growing importance as IPv6 adoption increases; often configured alongside A records.

CNAME Record (Canonical Name)

- Creates an alias pointing to another domain — e.g. `www.example.com → example.com`
- Cannot be used at zone apex (root domain); cannot coexist with other record types.

MX Record (Mail Exchange)

- Specifies mail servers for the domain with priority values (lower = higher priority).
- Example: `10 mail1.example.com, 20 mail2.example.com`

TXT Record (Text)

- Stores text information — SPF, DKIM, domain verification, site ownership proof.
- Example: `v=spf1 include:_spf.google.com ~all`

NS Record (Name Server)

- Delegates a subdomain to different name servers; specifies authoritative name servers for the zone.

- Example: `example.com → ns-123.awsdns-12.com`

SOA Record (Start of Authority)

- Contains zone metadata: primary name server, admin email, serial number, refresh/retry timings.
- Automatically managed by Route 53; only one SOA per zone.

PTR Record (Pointer)

- Reverse DNS lookup (IP to domain); important for email deliverability.
- Example: `1.2.0.192.in-addr.arpa → mail.example.com`

SRV Record (Service)

- Specifies location of services (SIP, XMPP, LDAP). Format: `priority weight port target`

CAA Record (Certification Authority Authorization)

- Specifies which CAs can issue certificates — prevents unauthorized certificate issuance.
- Example: `0 issue "letsencrypt.org"`

2.4 Time To Live (TTL)

TTL is the duration (in seconds) that DNS records are cached. Choosing the right TTL is a trade-off between performance and agility.

TTL Range	Advantages	Disadvantages
Short (60–300 s)	Faster failover; changes propagate rapidly	More DNS queries (higher cost); increased latency
Long (3600–86400 s)	Fewer DNS queries (lower cost); better cached performance	Slow propagation of changes; delayed failover

Recommended TTL Strategy

- Production stable records: `3600–7200 s` (1–2 hours)
- Pre-migration: `300 s` (5 minutes)
- During migration: `60 s` (1 minute)
- Post-migration: increase gradually

2.5 DNS Caching Behavior

DNS responses are cached at multiple levels, affecting how quickly changes propagate:

```
Browser Cache        → TTL: Varies (often ignores DNS TTL)
OS Cache             → TTL: Respects DNS TTL
Router/Gateway Cache → TTL: Respects DNS TTL
ISP Resolver Cache   → TTL: Respects DNS TTL (mostly)

Maximum Propagation Time = Longest Cache TTL
```

PRACTICAL NOTE

Even after reducing TTL to 300 s, full propagation can take longer than 5 minutes due to non-compliant resolvers. Plan accordingly.

3. Route 53 Hosted Zones

A hosted zone is a container for DNS records belonging to a domain — think of it as a database of DNS records for your domain.

3.1 Types of Hosted Zones

Attribute	Public Hosted Zone	Private Hosted Zone
Purpose	Internet-facing domains	Internal VPC resources
Access	Publicly resolvable from internet	Only from associated VPCs
Use Cases	Website domains, API endpoints, email servers	Internal apps, databases, microservices, dev environments
Cost (zone)	$0.50/month	$0.50/month
Query Cost	$0.40 per million queries	$0.40 per million queries
Requirements	Domain registration	VPC association, DNS resolution & hostnames enabled

3.2 Hosted Zone Architecture

```
Public Hosted Zone (example.com)
├── Name Servers (4 provided by Route 53)
│   ├── ns-123.awsdns-12.com
│   ├── ns-456.awsdns-45.co.uk
│   ├── ns-789.awsdns-78.org
│   └── ns-012.awsdns-01.net
├── Records
│   ├── A:     example.com      → 192.0.2.1
│   ├── A:     www.example.com  → 192.0.2.1
│   ├── CNAME: blog.example.com → blog-platform.com
│   ├── MX:    example.com      → mail.example.com
│   └── TXT:   example.com      → "verification-code"
└── Delegation
    └── NS: subdomain.example.com → other-name-servers
```

3.3 Split-View DNS (Hybrid Setup)

Route 53 supports split-view DNS where the same domain resolves differently for internal vs. external users:

```
Public Hosted Zone (example.com)
├── www.example.com → 203.0.113.1      (Public IP)
└── api.example.com → CloudFront distribution

Private Hosted Zone (example.com)
├── www.example.com  → 10.0.1.100      (Private IP)
├── api.example.com  → 10.0.2.50       (Internal ALB)
└── database.example.com → 10.0.3.10  (RDS endpoint)

External Users       → see public zone records
Internal VPC Users   → see private zone records (takes precedence)
```

Use Cases for Split-View DNS

- Internal services bypass the internet
- Lower latency for internal access
- Security (internal resources not exposed)
- Cost savings (no internet data transfer)

4. Routing Policies

Route 53's routing policies enable intelligent traffic distribution far beyond simple DNS resolution.

4.1 Simple Routing Policy

The most basic policy — one record with one or more IP addresses.

- Returns all values to the client; client chooses randomly.
- No health checking; no traffic distribution control.

```
www.example.com (A record)
├── 192.0.2.1
├── 192.0.2.2
└── 192.0.2.3
Client receives all three IPs, picks one randomly.
```

Best for: Single web server, static websites, development environments with no failover requirements.

4.2 Weighted Routing Policy

Distributes traffic across multiple resources based on assigned weights (0–255). Traffic % = (weight / sum of all weights) x 100.

Record	Weight	Traffic %
A	70	70%
B	20	20%
C	10	10%

Gradual Migration Strategy Example

Day	Old Weight	New Weight	Result
Day 1	100	0	100% old
Day 2	90	10	90% old / 10% new
Day 3	70	30	70% old / 30% new
Day 4	50	50	50/50 split

Day 5	20	80	20% old / 80% new
Day 6	0	100	100% new

Best for: Blue-green deployments, A/B testing, gradual traffic migration.

4.3 Latency-Based Routing Policy

Routes users to the region with lowest network latency. Route 53 uses historical latency data measured between user locations and AWS regions (not real-time per-query).

```
www.example.com (Latency policy)
├── us-east-1      → 52.1.2.3      (New York)
├── eu-west-1      → 34.240.5.6    (Ireland)
├── ap-southeast-1 → 13.228.7.8    (Singapore)
└── ap-south-1     → 13.126.9.10   (Mumbai)

User in London → eu-west-1 selected (10 ms vs 80 ms / 180 ms / 150 ms)
User in Tokyo  → ap-southeast-1 selected (75 ms vs 150 ms / 240 ms / 100 ms)
```

Best for: Global applications, multi-region deployments, international audiences.

4.4 Failover Routing Policy

Provides active-passive failover for high availability. A health check is required on the primary resource.

```
www.example.com (Failover policy)
├── Primary:   us-east-1 ALB (52.1.2.3)
│              └── Health Check: HTTP port 80, path /health
└── Secondary: us-west-2 ALB (34.208.4.5)
               └── Always available (no health check required)

Normal:    All traffic → us-east-1
Failure:   Automatic switch → us-west-2
Recovery:  Automatic switch back → us-east-1
```

Best for: Disaster recovery, active-passive architecture, high availability requirements.

4.5 Geolocation Routing Policy

Routes users based on their geographic location. Location precedence (most specific wins):

1. US State (e.g., California)

2. Country (e.g., United States)
3. Continent (e.g., North America)
4. Default location (fallback)

```
www.example.com (Geolocation policy)
├── Europe             → eu-west-1   (German website version)
├── Asia               → ap-southeast-1 (Asian market version)
├── United States      → us-east-1   (US version)
├── California (State) → us-west-1   (CA-specific)
└── Default            → us-east-1   (unmatched fallback)

Germany   → eu-west-1     | India      → ap-southeast-1
California → us-west-1    | New York   → us-east-1
Antarctica → us-east-1 (default)
```

> **IMPORTANT NOTES**
>
> Geolocation is based on the location of the DNS resolver, NOT the end user. VPN/proxy use can affect resolution. Always configure a Default location record.

Best for: Content localization, license restrictions, compliance/data residency, regional pricing.

4.6 Geoproximity Routing Policy

Routes based on geographic location of resources and users, with **bias** adjustment (–99 to +99). Requires Traffic Flow for configuration.

- Positive bias: expands the geographic region that routes to the resource.
- Negative bias: shrinks the region.
- For non-AWS resources: specify latitude/longitude manually.

```
www.example.com (Geoproximity policy)
├── us-east-1:  Bias +20  → expands coverage (attracts more traffic)
├── eu-west-1:  Bias  0   → neutral, standard proximity routing
└── ap-south-1: Bias -30  → shrinks coverage (test phase)
```

Best for: Load balancing with geographic preference, gradually shifting traffic to new regions.

4.7 Multivalue Answer Routing Policy

Returns up to 8 healthy records randomly in response to DNS queries. Unlike Simple routing, it performs health checks and excludes unhealthy records.

```
www.example.com (Multivalue policy)
├── Record 1: 192.0.2.1 (Healthy)
├── Record 2: 192.0.2.2 (Unhealthy) ← NOT returned
├── Record 3: 192.0.2.3 (Healthy)
├── Record 4: 192.0.2.4 (Healthy)
└── Record 5: 192.0.2.5 (Healthy)

Query Response: returns IPs 1, 3, 4, 5 — client picks one randomly
```

> **NOT A LOAD BALANCER REPLACEMENT**
>
> Multivalue lacks session affinity, SSL termination, and layer-7 routing. It is limited to DNS-level distribution only.

4.8 Routing Policy Comparison Matrix

Policy	Health Check	Complexity	Primary Use Case	Best For
Simple	No	Simplest	Basic DNS resolution	Dev / Test
Weighted	Yes	Low	A/B testing, traffic split	Migration
Latency	Yes	Medium	Global performance	Performance
Failover	Required	Low	High availability / DR	Uptime
Geolocation	Yes	Medium	Compliance, localization	Regional
Geoproximity	Yes	High	Fine-tune geographic routing	Advanced
Multivalue	Yes	Low	Simple load distribution	Basic HA

5. Health Checks

Health checks monitor resource availability and trigger routing changes when resources become unhealthy.

5.1 Health Check Types

Type 1: Endpoint Health Checks

Monitors an HTTP/HTTPS/TCP endpoint for response code, response time, and optionally response body content.

Parameter	Options / Value
Protocol	HTTP, HTTPS, TCP
Port	Any port (80, 443, custom)
Path	`/health`, `/api/status`
Interval	30 s (standard) or 10 s (fast)
Failure Threshold	3 consecutive failures (default)
HTTP healthy	2xx or 3xx response code
HTTPS healthy	SSL handshake success + 2xx/3xx
TCP healthy	Successful connection

String Matching (optional): verify the response body contains a specific string (e.g. `"status:ok"` in JSON).

Type 2: Calculated Health Checks

Monitors the status of other health checks using boolean logic (AND, OR, NOT). Useful for complex health scenarios.

```
Application Healthy =
  (Database Healthy AND Cache Healthy) OR Backup-DB Healthy
```

Configure a minimum threshold of healthy children out of a list of child health checks.

Type 3: CloudWatch Alarm Health Checks

Health status is derived from a CloudWatch alarm state — enabling custom health criteria based on any CloudWatch metric.

Alarm State	Health Status
OK	Healthy
ALARM	Unhealthy
Insufficient Data	Unhealthy

Example triggers: CPU utilization > 90%, 5xx error rate > 5%, DB connections > 1000, custom application metrics.

5.2 Health Check Locations & Consensus

- Route 53 checks from approximately **15 global locations** (Standard) or ~18 (Fast interval).
- Regions include: United States, Europe, Asia Pacific, South America, Africa, Middle East.
- **Consensus evaluation:** majority of checkers must report healthy (e.g., at least 8 of 15). Single checker failures do not trigger failover.

> **RECOMMENDATION**
>
> Configure your firewall to allow all Route 53 health checker IPs. Use CloudWatch metrics to monitor health check status over time.

5.3 Configuration Best Practices

Setting	Standard (30 s)	Fast (10 s)
Cost	~$0.50/month	~$1.00/month
Detection Time	~90 s (3 x 30 s)	~30 s (3 x 10 s)
Best For	Most scenarios	Critical apps needing rapid failover

Health Check Path Guidelines

Path	Assessment	Reason
`/` (homepage)	Avoid	Heavy processing, not a true health signal
`/health`	Good	Lightweight dedicated endpoint

`/api/health`	Best	Checks dependencies (DB, cache)

Endpoint requirements: fast response (< 2 s), check critical dependencies, return `200 OK` when healthy, `5xx` when unhealthy, do not cache health check responses.

6. Alias Records vs. CNAME Records

This is one of the most critical and frequently tested Route 53 concepts. The choice between Alias and CNAME has significant architectural and cost implications.

Feature	CNAME Record (Standard DNS)	Alias Record (Route 53-Specific)
Zone Apex	**Cannot use at root domain**	**Supported at zone apex**
Coexistence	**Cannot coexist with other records**	**Can coexist with other records**
Query Cost	Charged per query (2x lookups)	Free for queries to AWS resources
Target	Any domain name	AWS resources only
IP Updates	Manual	Automatic
Compatibility	Works with any DNS provider	Route 53-only feature

Supported Alias Targets (AWS Resources)

- CloudFront distributions
- Elastic Load Balancers (ALB, NLB, CLB)
- S3 website endpoints
- API Gateway
- VPC endpoints
- AWS Global Accelerator
- Another Route 53 record in the same hosted zone

Decision Tree

```
Need to route zone apex (root domain)?
├── YES
│   └── Pointing to AWS resource?
│       ├── YES → Use ALIAS
│       └── NO  → Cannot (DNS RFC restriction — no workaround)
└── NO (subdomain)
    └── Pointing to AWS resource?
        ├── YES → Prefer ALIAS (free queries, automatic IP)
        └── NO  → Use CNAME
```

Example Configurations

Record	Type	Target
`example.com`	Alias	CloudFront distribution
`www.example.com`	Alias	Application Load Balancer
`api.example.com`	Alias	API Gateway
`blog.example.com`	CNAME	hosting-platform.com

7. Traffic Flow & Traffic Policies

Traffic Flow provides a visual editor for building complex routing configurations that combine multiple routing policies.

7.1 Capabilities

- Drag-and-drop visual policy editor
- Combine multiple routing policies (geolocation, latency, failover, weighted, geoproximity)
- Complex decision trees with version control
- Reusable policies across hosted zones

7.2 Policy Components

```
Traffic Policy
├── Endpoints (targets: IP, ALB, etc.)
└── Rules (routing decisions)
    ├── Geolocation rules
    ├── Geoproximity rules
    ├── Latency rules
    ├── Failover rules
    └── Weighted rules
    (Combined with health checks)
```

7.3 Cost Considerations

Item	Cost
Policy Record	$50/month per policy record
Suitability	Expensive for simple scenarios; cost-effective for complex routing

7.4 When to Use Traffic Flow

Use Traffic Flow	Skip Traffic Flow
Complex multi-region architectures	Simple routing scenarios
Multiple routing policies combined	Cost-sensitive applications

Need visual representation	Single routing policy is sufficient
Frequently update routing logic	Static routing configuration
Reuse policies across domains	

7.5 Real-World Traffic Flow Example

Global E-commerce Site Requirements:

1. Route users to nearest region (latency-based)
2. Within each region, distribute traffic across AZs (weighted)
3. Failover to backup region if primary is unhealthy
4. Special handling for EU users (GDPR compliance)
5. Gradually migrate traffic to new infrastructure

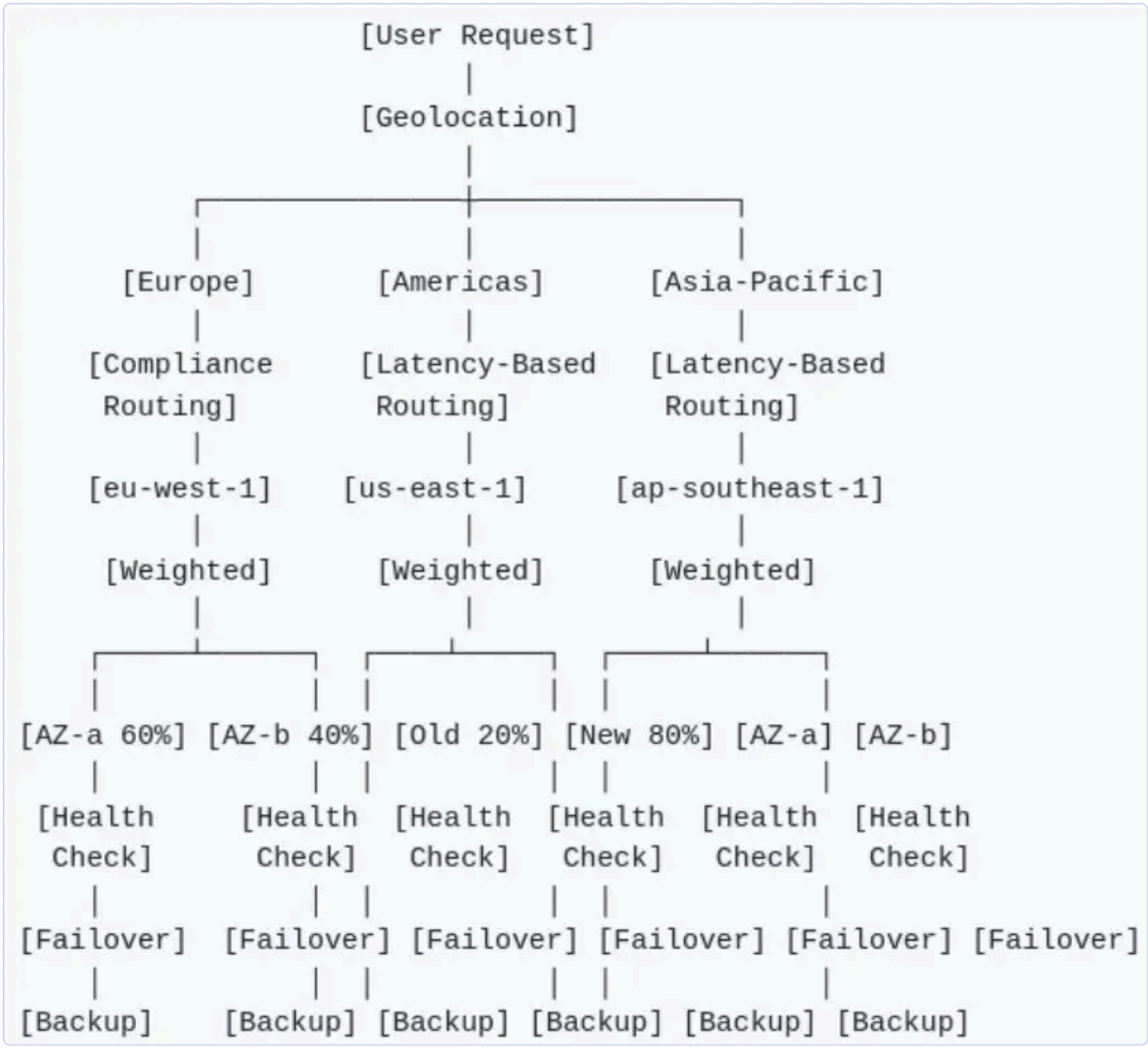


Figure 1: Traffic Flow visual policy editor — combining geolocation, latency, weighted, and failover routing for a global e-commerce platform.

> **KEY INSIGHT**
>
> This level of complexity cannot be achieved with individual simple routing policies. Traffic Flow is the only Route 53 mechanism that lets you compose multiple routing policies into a single decision tree.

8. DNSSEC (DNS Security Extensions)

DNSSEC adds cryptographic signatures to DNS records, preventing DNS spoofing and cache poisoning attacks.

8.1 The Problem DNSSEC Solves

Without DNSSEC:

1. User requests `www.bank.com`
2. Attacker intercepts DNS response
3. Attacker returns a fake IP (their phishing server)
4. User connects to the phishing site — credentials stolen

With DNSSEC: DNS records are digitally signed. Tampered records are detected and rejected via a chain of trust.

8.2 DNSSEC Record Types

Record	Purpose
RRSIG	Digital signature for a record set
DNSKEY	Public key used for signature verification
DS	Delegation Signer — links child zone to parent zone
NSEC / NSEC3	Authenticated denial of existence

8.3 Chain of Trust

```
Root Zone (signed by ICANN)
  └── .com TLD (signed by root)
        └── example.com (signed by .com)
              └── www.example.com (signed by example.com)

Each level verifies the signature of the next level
```

8.4 Enabling DNSSEC in Route 53

1. Enable signing in the Route 53 hosted zone.
2. Route 53 generates a KSK (Key Signing Key) — managed via AWS KMS.

3. Route 53 generates a ZSK (Zone Signing Key).
4. Add a DS record to the parent zone (registrar for .com).
5. DNSSEC chain of trust is established.

8.5 Key Management & Costs

Item	Detail
ZSK Rotation	Automatic
KSK Rotation	Manual (recommended yearly); zero-downtime process
DNSSEC Signing Cost	$0.50/month per hosted zone
KSK Storage (KMS)	$0.10/month per KSK
CloudWatch Alarm (KSK)	Free

8.6 When to Enable vs. Avoid DNSSEC

Enable DNSSEC	Avoid DNSSEC
Financial services	Development / test environments
Healthcare applications	Internal-only domains
E-commerce sites	Legacy system compatibility issues
Government systems	No security requirements
High-security / compliance mandates	Registrar doesn't support DS records

8.7 Monitoring DNSSEC

- Set up CloudWatch alarms for key rotation reminders.
- Test DNSSEC validation regularly from DNSSEC-aware resolvers.
- Monitor for signing errors; have a rollback plan ready.

9. Tips & Best Practices

TIP 1: LOWER TTL BEFORE CHANGES

Reduce TTL to 60 seconds at least 24 hours before making DNS changes. This ensures rapid propagation once the change is live and avoids long wait times caused by stale caches.

TIP 2: USE ALIAS RECORDS FOR AWS RESOURCES

Always use alias records (not CNAME) when pointing to CloudFront, ALB, S3, or API Gateway. Alias records are free, work at the zone apex, and automatically track IP changes.

TIP 3: IMPLEMENT HEALTH CHECKS FOR ALL FAILOVER

Never configure failover routing without health checks. Route 53 relies entirely on health check status to determine when to failover — without it, failover will never trigger.

TIP 4: TEST FAILOVER PROCEDURES REGULARLY

Simulate health check failures periodically to verify that failover behaves as expected. Do not wait until an actual outage to discover misconfiguration.

TIP 5: USE PRIVATE HOSTED ZONES FOR INTERNAL RESOURCES

Keep internal resource names (databases, microservices, internal APIs) in private hosted zones associated with your VPCs. Never expose internal DNS records publicly.

10. Pitfalls & Remedies

PITFALL 1: DNS PROPAGATION DELAYS

PROBLEM:

DNS changes appear to take hours to propagate globally despite low TTL settings.

ROOT CAUSE:

Old TTL was high; caches are still serving stale records. Non-compliant resolvers may ignore TTL.

SOLUTION:

Lower TTL 24–48 hours before changes and wait for the old TTL to expire before making changes. Verify with DNS propagation tools.

PITFALL 2: HEALTH CHECK FALSE POSITIVES

PROBLEM:

Health checks report unhealthy even though the resource is actually serving traffic correctly.

ROOT CAUSE:

Firewall blocking Route 53 health checker IPs; slow-responding health path; threshold too low.

SOLUTION:

Whitelist Route 53 health checker IP ranges in security groups/firewalls. Use a lightweight `/health` endpoint. Increase the failure threshold.

PITFALL 3: USING CNAME FOR ZONE APEX

PROBLEM:

Attempting to create a CNAME record for the root domain (e.g., `example.com`) results in a DNS RFC error.

ROOT CAUSE:

DNS RFC 1034 prohibits CNAME at the zone apex. This is a hard limit — not a Route 53 restriction.

SOLUTION:

Use Route 53 Alias records for all zone apex routing. Alias records behave like CNAME but are fully RFC-compliant.